

Prediction Algorithm Accuracy Report — v2

Post-Remediation: What Was Tried, What Shipped, What's Next

Sports Analytics Project

September 2026

Abstract

This is a follow-up to the initial backtest report (`prediction-accuracy-report.pdf`, same directory), which identified five recommendations for improving the platform's four prediction algorithms. All five were acted on. Two produced real, shipped improvements to system trustworthiness (an ongoing accuracy-monitoring script, and honest UI framing of the weaker margin prediction). One was a documentation fix. The remaining two — re-tuning Elo's constants and adding an opponent-defense adjustment to the player-point predictors — were properly tested against held-out data and *did not generalize*: both looked marginally better on the data used to select them, then performed no better (or slightly worse) on a chronological validation split. Per this project's existing practice of not presenting overfit results as reliable, neither was shipped. This report documents why, and — more usefully — identifies where the actual remaining headroom is, so future effort targets higher-leverage changes instead of repeating these two.

Contents

1	What Changed Since v1	1
2	Backtested Accuracy: Unchanged	2
3	Why the Two Tuning Attempts Came Back Negative	2
3.1	Elo constant re-tuning	2
3.2	Opponent-defense adjustment	3
4	Where the Real Headroom Is	3
5	Process Note	4

1 What Changed Since v1

Three of five were low-effort, unambiguous wins with no tradeoffs. The other two were genuine attempts at improving raw model accuracy, tested with the same discipline as everything else in this codebase (walk-forward, leak-free, validated on held-out data) — and came back negative. That is a normal and expected outcome in applied modeling, not a sign anything was done wrong; §3 explains specifically why each one was unlikely to work *before* even seeing the results, with the benefit of hindsight, and §4 gives concrete, higher-leverage alternatives.

Recommendation	Outcome	Detail
#1 Accuracy-tracking job	Shipped	<code>check_accuracy.py</code> in <code>apps/predictor</code> — read-only, re-runnable, reuses <code>elo.py/four_factors.py</code> directly
#2 Opponent-adjustment for player predictors	Not shipped	Backtested; edge too small and didn't hold on a validation split (§3.2)
#3 Re-tune Elo constants	Not shipped	Grid-searched; train-split winner validated worse than the defaults (§3.1)
#4 De-emphasize Four Factors margin in UI	Shipped	<code>GameDetailPage.tsx</code> , <code>PredictionsPage.tsx</code> — margin is now visually secondary with an explanatory tooltip
#5 Fix stale “seed data” comments	Shipped	<code>four_factors.py</code> and its README — now describe the live full-season state

Table 1: Disposition of all five v1 recommendations.

2 Backtested Accuracy: Unchanged

Because neither of the two accuracy-focused experiments was shipped, and the three shipped changes were documentation/monitoring/UI-only, **every backtested accuracy number from v1 is still exactly correct today** — re-run in full against the live database while writing this report and confirmed identical:

Model	Metric	Value (unchanged from v1)
Elo win probability	Brier score	0.2146
Elo win probability	Accuracy	65.2% (vs. 55.5% naive)
Four Factors margin	MAE	12.52 pts (vs. 13.36 naive)
Fantasy points	MAE	7.74 (vs. 8.25 naive)
Scorer points	MAE	4.45 (vs. 4.64 naive)

Table 2: All four models' backtested accuracy is unchanged from the v1 report — see that report for full methodology, calibration tables, and permutation test results, all still current.

3 Why the Two Tuning Attempts Came Back Negative

3.1 Elo constant re-tuning

Method. A chronological 70/30 train/validation split (first 924 games to grid-search `HOME_COURT_ADVANTAGE_ELO` and `K_FACTOR` by Brier score, last 397 games held out purely to check generalization — a random split would be invalid here since Elo's rating updates are inherently sequential).

Result. The train-split grid search's winner (HCA=30, K=30) beat the literature-default (HCA=75, K=20) on the training data (Brier 0.2233 vs. roughly 0.2251), but validated *worse* on the held-out split: 0.1839 vs. 0.1829 Brier. That is the textbook signature of overfitting — a configuration that looks better only because it was chosen using the exact data it's being judged on.

Why, in hindsight, this was unlikely to work. The training-split Brier surface across the

entire tested HCA range (0 to 100) was quite flat — 0.2233 to 0.2251, a spread of only 0.0018. When a hyperparameter surface is that flat, the “optimal” point found by a grid search is mostly noise: small sampling fluctuations in which specific games fell in the training window are enough to shift where the minimum lands. A single NBA season (1,321 games, effectively ~ 44 games per team-pair-orientation once split by home/away) is simply not a lot of independent evidence about a single global home-court constant — the literature-derived defaults were themselves fit on vastly more games across many seasons, so it would be mildly surprising if one season’s worth of this project’s data could improve on them.

3.2 Opponent-defense adjustment

Method. The simplest possible opponent adjustment — scale a player’s baseline recency-weighted prediction by the ratio of the upcoming opponent’s leak-free running points-allowed average to the leaguewide average at that point in the season — backtested against the full season, then re-checked on a chronological 30% validation-only slice.

Result. Full-season MAE improved by 0.006 (fantasy points) and 0.002 (raw points) — both negligible. On the validation-only slice, the fantasy-points edge shrank further and the raw-points edge *flipped slightly negative* (-0.0001).

Why, in hindsight, this was unlikely to work. Team-level points-allowed only spans roughly $\pm 9\%$ across the entire league (106.9 to 126.0 points allowed per game, league mean 115.3). A $\pm 9\%$ multiplicative nudge is small next to a single player’s game-to-game scoring variance (RMSE ≈ 10 fantasy points / ≈ 6 raw points) — the adjustment was always going to be a small correction relative to the noise it’s competing with. This isn’t evidence that opponent strength doesn’t matter to basketball outcomes; it’s evidence that *team-wide* points-allowed is too coarse a proxy for it at the individual-player level. A stronger signal (e.g. points allowed to players at the same position specifically, or a proper defensive rating that adjusts for pace) might carry more information, but that’s a materially different, larger effort than the one-line adjustment tested here — see §4.

4 Where the Real Headroom Is

Both negative results above share a pattern worth naming explicitly: **they were both attempts to extract more signal from data the models already have**, just recombined differently (better-tuned constants, a coarser opponent proxy). That data was already close to fully exploited by the existing simple models. The remaining, larger opportunities are in **data the models don’t have access to at all yet** — these are not guaranteed to work either, but they target genuinely unused information, which is a fundamentally different (and more promising) kind of bet than re-weighting existing inputs.

1. **Minutes / role-change signal for the player-point predictors.** Neither `predict.py` nor `game-detail.service.ts`’s predictor has any feature for playing time — a player whose minutes just changed (returning from injury, moved into the starting lineup, a coach’s rotation change) will be mispredicted by a pure box-score average until enough new games accumulate to catch up organically. This is a bigger lever than opponent adjustment because minutes changes are a much larger relative swing in a player’s output than an opponent’s defensive quality is. Concretely: track a player’s own minutes-per-game trend (the same recency-weighting technique already used for points would work directly for minutes) and either use it as an additional regression feature or predict a per-minute rate and multiply by a minutes

forecast.

2. **Offensive/defensive rebound split for Four Factors.** This is the one item from the original report requiring a real (if modest) ingestion/schema change: `PlayerGameStat` currently tracks only total rebounds, not an offensive/defensive split, which is why the current model uses 3 of Oliver’s 4 factors and explicitly documents the gap. Adding that split is the single most direct way to close some of the Four Factors margin model’s remaining error, since it’s a genuinely new dimension of information (not a recombination of what’s already there, unlike the two experiments above).
3. **More seasons of data for Elo.** §3.1’s finding was specifically that one season isn’t enough evidence to beat a good literature prior — that is a data-volume problem, not a methodology problem. Re-attempting the same grid search once 2-3 seasons are loaded is likely to behave very differently (a much less flat, better-identified Brier surface), and is a “wait and re-run,” not a “design something new,” item.
4. **Home/away split for the player-point predictors.** Simple to add (the data already exists on every `Game` row, same as opponent identity was), well-established in the wider basketball analytics literature, and untested by any of the work in this report or its predecessor — a reasonable next candidate to backtest, though given §3.2’s result, it should be validated the same way (train/validation split, not just full-season MAE) before being treated as a win.

5 Process Note

Everything in this report — including the two negative results — was produced using the same walk-forward, leak-free, held-out-validation discipline the underlying prediction code itself already enforces (see `four_factors.py` and `games.e2e-spec.ts`’s existing data-leakage tests). Two specific ideas not generalizing is a normal rate of return for this kind of iteration, not a signal that the approach is wrong — the value was in finding out *before* shipping either change, via the same discipline that caught a real future-data leak in this codebase once before (documented in `four_factors.py`’s own module docstring). The three items in §4 are concrete next bets, ranked by how much genuinely new information they’d add rather than by how easy they’d be to implement.